Package ‘afcestDataCheck’

February 28, 2026

Title Survey Data Quality Checks for Development Projects

Version 0.3.0

Description Comprehensive high-frequency data quality checks for agricultural, health, and socioeconomic surveys. 54 check functions across 10 categories: identification, metadata, completeness, outliers, enumerator performance, timing, GPS validation, fabrication detection, logic consistency, and text quality. Includes orchestrator (run_all_checks), Excel/CSV export, and multi-platform normalizer for KoBoToolbox, SurveyCTO, ODK Central, Survey Solutions, and CSPro.

License MIT + file LICENSE

Encoding UTF-8

LazyData false

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.1

Imports cli (>= 3.6.0),
dplyr (>= 1.1.0),
rlang (>= 1.1.0),
stats,
yaml

Suggests DBI,
gt,
httr2,
knitr,
openxlsx2,
rmarkdown,
robotoolbox,
RSQLite,
rsurveycto,
ruODK,
sf,
SurveySolutionsAPI,
testthat (>= 3.0.0),
withr

VignetteBuilder knitr

Config/testthat/edition 3

Contents

`apply_corrections` *Apply corrections from a correction log to survey data*

Description

Q.01: For each row in the correction log, find the matching record in data, verify the old value matches, and apply the new value. Returns the corrected data along with a summary of applied and skipped corrections.

Usage

```
apply_corrections(
  data,
  correction_log,
  id_col = "id",
  variable_col = "variable",
  old_value_col = "old_value",
  new_value_col = "new_value",
  ...
)
```

Arguments

`data`	Data frame to correct
`correction_log`	Data frame with one row per correction. Must contain columns for record ID, variable name, old value, and new value.
`id_col`	Character. Name of the ID column in both data and correction_log (default "id")
`variable_col`	Character. Column in correction_log containing the variable name to correct (default "variable")
`old_value_col`	Character. Column in correction_log containing the expected current value (default "old_value")
`new_value_col`	Character. Column in correction_log containing the replacement value (default "new_value")
`...`	Reserved for future use

Details

This is a data modification function, NOT a check function. It does not return a check_result object.

Value

A list with components:

corrected_data The modified data frame

n_applied Integer. Number of corrections successfully applied

n_skipped Integer. Number of corrections that could not be applied

skipped_log A tibble with columns: row_index, id, variable, reason (describing why the correction was skipped)

`bind_check_results` *Combine multiple check results into a summary tibble*

Description

Combine multiple check results into a summary tibble

Usage

```
bind_check_results(...)
```

Arguments

`...`	check_result objects or lists of check_result objects

Value

A tibble summarizing all checks

cfg_get *Get a config value with a default fallback*

Description

Get a config value with a default fallback

Usage

```
cfg_get(cfg, ..., default = NULL)
```

Arguments

`cfg`	adc_config object
`...`	Path components (e.g., "checks", "outliers", "method")
`default`	Default value if path not found

Value

The config value or default

check_age_date_consistency *Check age-date of birth consistency*

Description

F.09: Flag records where reported age does not match the age computed from date of birth.

Usage

```
check_age_date_consistency(
  data,
  id_col,
  age_col,
  dob_col,
  reference_date = Sys.Date(),
  tolerance_years = 1,
  ...
)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`age_col`	Character. Name of the reported age column
`dob_col`	Character. Name of the date of birth column
`reference_date`	Date. Reference date for computing age (default Sys.Date())
`tolerance_years`	Numeric. Acceptable discrepancy in years (default 1)
`...`	Reserved for future use

Value

A check_result object

`check_all_missing_variables`
Check for variables with 100% missing values

Description

C.04: Variables with zero non-missing values.

Usage

```
check_all_missing_variables(data, ...)
```

Arguments

`data`	Data frame
`...`	Reserved

Value

A check_result object

`check_backcheck_by_enumerator`
Check back-check error rates by enumerator

Description

J.06: Compute per-enumerator error rates across all comparison columns and flag enumerators whose error rate is more than threshold_sd standard deviations above the team mean.

Usage

```
check_backcheck_by_enumerator(
  data,
  id_col,
  enum_col,
  backcheck_data,
  bc_id_col,
  compare_cols,
  threshold_sd = 2,
  ...
)
```

Arguments

`data`	Data frame of original survey submissions
`id_col`	Character. Primary key column in original data
`enum_col`	Character. Enumerator ID column in original data
`backcheck_data`	Data frame of back-check submissions
`bc_id_col`	Character. ID column in backcheck_data matching data
`compare_cols`	Character vector. Columns to compare between original and back-check data (must exist in both)
`threshold_sd`	Numeric. Number of SDs above team mean to trigger flag (default 2)
`...`	Reserved for future use

Value

A check_result object

`check_backcheck_coverage`
Check back-check coverage

Description

J.01: Verify that a sufficient percentage of original surveys have been back-checked. If strata are provided, coverage is computed per stratum.

Usage

```
check_backcheck_coverage(
  data,
  id_col,
  backcheck_data,
  bc_id_col,
  strata_col = NULL,
  min_coverage = 0.1,
  ...
)
```

Arguments

`data`	Data frame of original survey submissions
`id_col`	Character. Name of the primary key column in original data
`backcheck_data`	Data frame of back-check submissions
`bc_id_col`	Character. Name of the ID column in backcheck_data that matches records in data
`strata_col`	Character or NULL. Column in data for stratified coverage (e.g., region, district). Default NULL computes overall coverage only.
`min_coverage`	Numeric. Minimum required coverage rate (default 0.10)
`...`	Reserved for future use

Value

A check_result object

---

check_backcheck_t1_match *Check Type 1 variable matches in back-checks*

Description

J.02: Type 1 variables (e.g., gender, location) should NEVER change between the original survey and the back-check. Any mismatch signals a serious data quality issue.

Usage

```
check_backcheck_t1_match(data, id_col, backcheck_data, bc_id_col, t1_cols, ...)
```

Arguments

`data`	Data frame of original survey submissions
`id_col`	Character. Primary key column in original data
`backcheck_data`	Data frame of back-check submissions
`bc_id_col`	Character. ID column in backcheck_data matching data
`t1_cols`	Character vector. Type 1 variable names (must exist in both data and backcheck_data)
`...`	Reserved for future use

Value

A check_result object

---

check_backcheck_t2_match *Check Type 2 variable matches in back-checks*

Description

J.03: Type 2 variables can legitimately change but are used to assess enumerator quality. High error rates indicate potential enumerator problems.

Usage

```
check_backcheck_t2_match(
  data,
  id_col,
  backcheck_data,
  bc_id_col,
  t2_cols,
  max_error_rate = 0.1,
  enum_col = NULL,
  ...
)
```

Arguments

`data`	Data frame of original survey submissions
`id_col`	Character. Primary key column in original data
`backcheck_data`	Data frame of back-check submissions
`bc_id_col`	Character. ID column in backcheck_data matching data
`t2_cols`	Character vector. Type 2 variable names (must exist in both data and backcheck_data)
`max_error_rate`	Numeric. Maximum acceptable per-enumerator error rate (default 0.10)
`enum_col`	Character or NULL. Enumerator column in original data. If NULL, per-enumerator analysis is skipped.
`...`	Reserved for future use

Value

A check_result object

`check_backcheck_t3_match` *Check Type 3 variable changes in back-checks*

Description

J.04: Type 3 variables are expected to change between original and back-check (e.g., satisfaction, attitude). This check is purely informational and computes change rates per variable.

Usage

```
check_backcheck_t3_match(data, id_col, backcheck_data, bc_id_col, t3_cols, ...)
```

Arguments

`data`	Data frame of original survey submissions
`id_col`	Character. Primary key column in original data
`backcheck_data`	Data frame of back-check submissions
`bc_id_col`	Character. ID column in backcheck_data matching data
`t3_cols`	Character vector. Type 3 variable names (must exist in both data and backcheck_data)
`...`	Reserved for future use

Value

A check_result object (informational, n_flagged = 0)

check_benford_first_digit

Check Benford's Law first-digit distribution

Description

M.01: Apply Benford's Law first-digit test on a numeric column. Computes the observed first-digit distribution (digits 1-9) and compares to the expected Benford distribution using a chi-squared goodness-of-fit test. Flags the VARIABLE (not individual records) if the distribution deviates significantly from Benford's Law.

Usage

```
check_benford_first_digit(data, id_col, num_col, alpha = 0.05, ...)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`num_col`	Character. Name of the numeric column to test
`alpha`	Numeric. Significance level for the chi-squared test (default 0.05)
`...`	Reserved for future use

Value

A check_result object

check_benford_second_digit

Check Benford's Law second-digit distribution

Description

M.02: Apply Benford's Law second-digit test on a numeric column. Computes the observed second-digit distribution (digits 0-9) and compares to the expected second-digit Benford distribution using a chi-squared test.

Usage

```
check_benford_second_digit(data, id_col, num_col, alpha = 0.05, ...)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`num_col`	Character. Name of the numeric column to test
`alpha`	Numeric. Significance level for the chi-squared test (default 0.05)
`...`	Reserved for future use

Details

Expected: P(d2) = sum over d1=1..9 of log10(1 + 1/(10*d1 + d2))

Value

A check_result object

`check_collection_window` *Check collection window*

Description

F.02: Flag surveys collected outside the expected date window.

Usage

```
check_collection_window(data, id_col, date_col, start_date, end_date, ...)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`date_col`	Character. Name of the column containing collection dates
`start_date`	Character or Date. Start of the valid collection window
`end_date`	Character or Date. End of the valid collection window
`...`	Reserved for future use

Value

A check_result object

`check_consent` *Check consent status*

Description

A.06: Flag records where consent was not given.

Usage

```
check_consent(data, id_col, consent_col, consent_value = 1, ...)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`consent_col`	Character. Name of the consent column
`consent_value`	Value indicating consent was given (default 1)
`...`	Reserved for future use

Value

A check_result object

`check_correction_log` *Validate a correction log before applying it*

Description

Q.02: Check a correction log for potential issues without modifying data. Flags entries where the ID does not exist in data, the variable does not exist in data, or the old value does not match the current value.

Usage

```
check_correction_log(
  data,
  correction_log,
  id_col = "id",
  variable_col = "variable",
  old_value_col = "old_value",
  ...
)
```

Arguments

`data`	Data frame to validate corrections against
`correction_log`	Data frame with one row per correction
`id_col`	Character. Name of the ID column (default "id")
`variable_col`	Character. Column in correction_log containing the variable name (default "variable")
`old_value_col`	Character. Column in correction_log containing the expected current value (default "old_value")
`...`	Reserved for future use

Value

A check_result object

check_custom_logic *Check custom logic condition*

Description

G.12: Evaluate a user-defined expression and flag records where the condition evaluates to TRUE (indicating a violation).

Usage

```
check_custom_logic(
  data,
  id_col,
  condition_expr,
  description = "Custom logic check",
  ...
)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`condition_expr`	Character. An R expression string that evaluates to a logical vector using column names from data. TRUE means violation.
`description`	Character. Human-readable description of the check (default "Custom logic check")
`...`	Reserved for future use

Value

A check_result object

check_device_consistency *Check device consistency per enumerator*

Description

N.08: Flag enumerators who used more than one device during data collection. Groups by enumerator and counts distinct device identifiers. Enumerators with more than one device are flagged.

Usage

```
check_device_consistency(data, id_col, enum_col, device_col, ...)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`enum_col`	Character. Name of the enumerator ID column
`device_col`	Character. Name of the device identifier column
`...`	Reserved for future use

Value

A check_result object

`check_digit_preference` *Check terminal digit preference (digit heaping)*

Description

M.03: Check the last (terminal) digit distribution of a numeric column. Under no digit preference, terminal digits should be roughly uniformly distributed across 0-9. Uses a chi-squared goodness-of-fit test against the uniform distribution.

Usage

```
check_digit_preference(data, id_col, num_col, alpha = 0.05, ...)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`num_col`	Character. Name of the numeric column to test
`alpha`	Numeric. Significance level for the chi-squared test (default 0.05)
`...`	Reserved for future use

Value

A check_result object

`check_dk_rate` *Check don't-know response rate by enumerator*

Description

B.03: Flag enumerators with an unusually high rate of "don't know" responses (more than threshold_sd SDs above team mean).

Usage

```
check_dk_rate(
  data,
  id_col,
  enum_col,
  dk_value = "dk",
  check_cols = NULL,
  threshold_sd = 2,
  ...
)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`enum_col`	Character. Name of the enumerator ID column
`dk_value`	Character. The "don't know" string value (default "dk")
`check_cols`	Character vector. Columns to inspect; if NULL, all columns except id_col and enum_col are used
`threshold_sd`	Numeric. Number of SDs above team mean to trigger flag (default 2)
`...`	Reserved for future use

Details

A cell is counted as DK if it matches dk_value (case-insensitive) or equals -99, -88, or -77.

Value

A check_result object

check_duplicate_fingerprint

Check for fingerprint duplicates (different IDs, same quasi-identifiers)

Description

A.02: Records with different IDs but matching GPS, phone, or name.

Usage

```
check_duplicate_fingerprint(data, id_col, quasi_ids, ...)
```

Arguments

`data`	Data frame
`id_col`	Primary key column
`quasi_ids`	Character vector of quasi-identifier column names
`...`	Reserved

Value

A check_result object

check_duplicate_ids

Check for duplicate survey IDs

Description

A.01: Finds records sharing the same primary key.

Usage

```
check_duplicate_ids(data, id_col, ...)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`...`	Reserved for future use

Value

A check_result object

check_duplicate_response_patterns

Check for duplicate response patterns across surveys

Description

M.07: For each pair of surveys, compute the proportion of matching responses across check_cols. Flag pairs exceeding similarity_threshold. Uses fingerprint hashing for efficiency: only computes pairwise similarity within groups sharing the same hash.

Usage

```
check_duplicate_response_patterns(
  data,
  id_col,
  check_cols,
  similarity_threshold = 0.95,
  ...
)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`check_cols`	Character vector. Columns to compare across surveys
`similarity_threshold`	Numeric. Proportion of matching responses to trigger a flag (default 0.95)
`...`	Reserved for future use

Value

A check_result object

check_duration_by_hh_size

Check survey duration by household size

Description

F.06: Flag surveys that are shorter than expected given the household size.

Usage

```
check_duration_by_hh_size(
  data,
  id_col,
  duration_col,
  hh_size_col,
  min_minutes_per_member = 3,
  ...
)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`duration_col`	Character. Name of the duration column (in minutes)
`hh_size_col`	Character. Name of the household size column
`min_minutes_per_member`	Numeric. Minimum expected minutes per HH member (default 3)
`...`	Reserved for future use

Value

A check_result object

`check_enumerator_duration` *Check enumerator survey duration*

Description

B.02: Flag enumerators whose mean survey duration is suspiciously low (more than threshold_sd standard deviations below the team mean).

Usage

```
check_enumerator_duration(
  data,
  id_col,
  enum_col,
  duration_col,
  threshold_sd = 2,
  ...
)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`enum_col`	Character. Name of the enumerator ID column
`duration_col`	Character. Name of the survey duration column (numeric, in minutes)
`threshold_sd`	Numeric. Number of SDs below team mean to trigger flag (default 2)
`...`	Reserved for future use

Value

A check_result object

`check_enumerator_productivity`
Check enumerator daily productivity

Description

B.01: Flag enumerator-days where survey count exceeds the maximum daily limit.

Usage

```
check_enumerator_productivity(
  data,
  id_col,
  enum_col,
  date_col,
  max_daily = 10,
  ...
)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`enum_col`	Character. Name of the enumerator ID column
`date_col`	Character. Name of the survey date column (parsed with as.Date())
`max_daily`	Integer. Maximum expected surveys per enumerator per day (default 10)
`...`	Reserved for future use

Value

A check_result object

`check_enumerator_time_gap`
Check time gaps between consecutive surveys by enumerator

Description

B.08: Flag surveys where the gap from the previous survey start time is impossibly short (less than min_gap_minutes).

Usage

```
check_enumerator_time_gap(
  data,
  id_col,
  enum_col,
  start_time_col,
  min_gap_minutes = 5,
  ...
)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`enum_col`	Character. Name of the enumerator ID column
`start_time_col`	Character. Name of the survey start time column (parsed with as.POSIXct())
`min_gap_minutes`	Numeric. Minimum expected gap in minutes (default 5)
`...`	Reserved for future use

Value

A check_result object

`check_field_comments` *Check field comments for supervisor review*

Description

N.04: Extract and list all non-empty field comments. This is informational: comments need human review and are flagged for supervisor attention.

Usage

```
check_field_comments(data, id_col, comment_col, ...)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`comment_col`	Character. Name of the comment column
`...`	Reserved for future use

Value

A check_result object

check_form_version — *Check form version*

Description

A.07: Flag records using an outdated form version.

Usage

```
check_form_version(data, id_col, version_col, expected_version, ...)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`version_col`	Character. Name of the form version column
`expected_version`	Character or numeric. The expected (current) form version
`...`	Reserved for future use

Value

A check_result object

check_future_dates — *Check for future dates*

Description

F.03: Flag records with dates in the future relative to a reference date.

Usage

```
check_future_dates(data, id_col, date_col, reference_date = Sys.Date(), ...)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`date_col`	Character. Name of the column containing dates
`reference_date`	Date. Reference date to compare against (default Sys.Date())
`...`	Reserved for future use

Value

A check_result object

check_gps_accuracy — *Check GPS accuracy*

Description

E.03: Flag GPS readings with accuracy greater than a threshold.

Usage

```
check_gps_accuracy(data, id_col, accuracy_col, max_accuracy = 50, ...)
```

Arguments

data	Data frame of survey submissions
id_col	Character. Name of the primary key column
accuracy_col	Character. Name of the GPS accuracy column (metres)
max_accuracy	Numeric. Maximum acceptable accuracy in metres (default 50)
...	Reserved for future use

Value

A check_result object

check_gps_altitude — *Check GPS altitude plausibility*

Description

E.10: Flag altitude values outside a plausible range.

Usage

```
check_gps_altitude(
  data,
  id_col,
  altitude_col,
  min_alt = -500,
  max_alt = 6000,
  ...
)
```

Arguments

data	Data frame of survey submissions
id_col	Character. Name of the primary key column
altitude_col	Character. Name of the GPS altitude column (metres)
min_alt	Numeric. Minimum plausible altitude in metres (default -500)
max_alt	Numeric. Maximum plausible altitude in metres (default 6000)
...	Reserved for future use

Value

A check_result object

---

check_gps_boundary *Check GPS boundary*

Description

E.01: Flag GPS points that fall outside a specified boundary. Accepts either a bounding box (named list) or an sf polygon. Falls back to valid geographic range [-90, 90] for lat and [-180, 180] for lon.

Usage

```
check_gps_boundary(
  data,
  id_col,
  lat_col,
  lon_col,
  boundary_bbox = NULL,
  boundary_sf = NULL,
  ...
)
```

Arguments

data	Data frame of survey submissions
id_col	Character. Name of the primary key column
lat_col	Character. Name of the latitude column
lon_col	Character. Name of the longitude column
boundary_bbox	Named list with xmin, xmax, ymin, ymax (lon/lat)
boundary_sf	An sf polygon object for point-in-polygon checks
...	Reserved for future use

Value

A check_result object

check_gps_centroid_distance

Check GPS distance from cluster centroid

Description

E.02: Flag GPS points that are more than a specified distance from their cluster centroid. Uses the Haversine formula.

Usage

```
check_gps_centroid_distance(
  data,
  id_col,
  lat_col,
  lon_col,
  cluster_col,
  centroid_data = NULL,
  max_distance_km = 10,
  ...
)
```

Arguments

data	Data frame of survey submissions
id_col	Character. Name of the primary key column
lat_col	Character. Name of the latitude column
lon_col	Character. Name of the longitude column
cluster_col	Character. Name of the cluster/EA column
centroid_data	Optional data frame with pre-computed centroids. Must contain columns matching cluster_col, lat_col, and lon_col.
max_distance_km	Numeric. Maximum distance in km from centroid (default 10)
...	Reserved for future use

Value

A check_result object

check_gps_clustering *Check excessive GPS clustering*

Description

E.08: Flag cases where too many surveys share the exact same GPS coordinates (rounded to 4 decimal places, approximately 11m), which may indicate data fabrication.

Usage

```
check_gps_clustering(
  data,
  id_col,
  lat_col,
  lon_col,
  enum_col = NULL,
  max_same_point = 3,
  ...
)
```

Arguments

data	Data frame of survey submissions
id_col	Character. Name of the primary key column
lat_col	Character. Name of the latitude column
lon_col	Character. Name of the longitude column
enum_col	Character or NULL. Enumerator ID column. If provided, only flags clusters where all surveys are from the same enumerator.
max_same_point	Integer. Maximum number of surveys allowed at the same GPS point before flagging (default 3)
...	Reserved for future use

Value

A check_result object

check_gps_duplicates *Check GPS duplicate coordinates*

Description

E.04: Flag GPS coordinates that are identical or within a minimum distance of each other across different survey IDs.

Usage

```
check_gps_duplicates(data, id_col, lat_col, lon_col, min_distance = 1e-04, ...)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`lat_col`	Character. Name of the latitude column
`lon_col`	Character. Name of the longitude column
`min_distance`	Numeric. Minimum distance in degrees below which points are considered duplicates (default 0.0001, approx 11m)
`...`	Reserved for future use

Value

A check_result object

`check_gps_null_island` *Check GPS null island and default values*

Description

E.07: Flag GPS coordinates at (0, 0), or with lat/lon equal to known default or placeholder values (0, 999, -999, 9999).

Usage

```
check_gps_null_island(data, id_col, lat_col, lon_col, ...)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`lat_col`	Character. Name of the latitude column
`lon_col`	Character. Name of the longitude column
`...`	Reserved for future use

Value

A check_result object

check_gps_swap — *Check for swapped latitude and longitude*

Description

E.05: Detect cases where latitude and longitude appear to be swapped. If an expected bounding box is provided, checks whether swapping the coordinates would place the point inside the expected area. Without a bounding box, flags rows where abs(lat) > 90 (impossible value that may be a longitude).

Usage

```
check_gps_swap(data, id_col, lat_col, lon_col, expected_bbox = NULL, ...)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`lat_col`	Character. Name of the latitude column
`lon_col`	Character. Name of the longitude column
`expected_bbox`	Optional named list with lat_min, lat_max, lon_min, lon_max defining the expected study area
`...`	Reserved for future use

Value

A check_result object

check_hard_range — *Check values against hard range constraints*

Description

D.04: Flag values outside a fixed [min_val, max_val] range.

Usage

```
check_hard_range(data, id_col, num_col, min_val = -Inf, max_val = Inf, ...)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`num_col`	Character. Name of the numeric column to check
`min_val`	Numeric. Minimum allowed value (default -Inf)
`max_val`	Numeric. Maximum allowed value (default Inf)
`...`	Reserved for future use

Value

A check_result object

`check_hh_composition` *Check household composition consistency*

Description

G.01: Flag records with implausible household size or mismatch between reported size and roster count.

Usage

```
check_hh_composition(data, id_col, hh_size_col, roster_count_col = NULL, ...)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`hh_size_col`	Character. Name of the household size column
`roster_count_col`	Character or NULL. Name of the roster count column (default NULL)
`...`	Reserved for future use

Value

A check_result object

`check_icc_enumerator` *Check intraclass correlation coefficient by enumerator*

Description

M.04: Compute one-way ICC for each numeric variable, grouping by enumerator. High ICC means enumerator identity explains a large share of variance, which may indicate fabrication or systematic enumerator bias.

Usage

```
check_icc_enumerator(data, id_col, enum_col, num_cols, max_icc = 0.15, ...)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`enum_col`	Character. Name of the enumerator ID column
`num_cols`	Character vector. Numeric columns to compute ICC for
`max_icc`	Numeric. Maximum acceptable ICC (default 0.15)
`...`	Reserved for future use

Details

ICC = (MSB - MSW) / (MSB + (n_avg - 1) * MSW)

Value

A check_result object

`check_id_format` *Check ID format against expected pattern*

Description

A.04: Flag IDs that do not match the expected regex pattern.

Usage

```
check_id_format(data, id_col, pattern, ...)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`pattern`	Character. Regular expression pattern that valid IDs must match
`...`	Reserved for future use

Value

A check_result object

`check_id_in_sample` *Check IDs against sampling frame*

Description

A.05: IDs not in the master sampling list.

Usage

```
check_id_in_sample(data, id_col, sampling_frame, ...)
```

Arguments

`data`	Data frame
`id_col`	Primary key column
`sampling_frame`	Character vector of valid IDs from master list
`...`	Reserved

Value

A check_result object

`check_income_expenditure` *Check income-expenditure ratio*

Description

G.04: Flag records where expenditure exceeds a plausible ratio of income.

Usage

```
check_income_expenditure(
  data,
  id_col,
  income_col,
  expenditure_col,
  max_ratio = 5,
  ...
)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`income_col`	Character. Name of the income column
`expenditure_col`	Character. Name of the expenditure column
`max_ratio`	Numeric. Maximum acceptable expenditure/income ratio (default 5)
`...`	Reserved for future use

Value

A check_result object

`check_interview_completed` *Check interview completion status*

Description

A.09: Flag records where the interview was not completed.

Usage

```
check_interview_completed(
  data,
  id_col,
  status_col,
  complete_value = "complete",
  ...
)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`status_col`	Character. Name of the interview status column
`complete_value`	Character. Value indicating a completed interview (default "complete")
`...`	Reserved for future use

Value

A check_result object

`check_missing_by_enumerator` *Check missing rate by enumerator*

Description

C.02: Flag enumerators with unusually high missing rates.

Usage

```
check_missing_by_enumerator(data, id_col, enum_col, threshold_sd = 2, ...)
```

Arguments

`data`	Data frame
`id_col`	Primary key column
`enum_col`	Enumerator ID column
`threshold_sd`	Numeric. Number of SD above team mean to flag (default 2)
`...`	Reserved

Value

A check_result object

check_missing_by_variable *Check missing rate by variable*

Description

C.01: Compute missing rate per variable and flag those above threshold.

Usage

```
check_missing_by_variable(
  data,
  id_col,
  threshold = 0.05,
  exclude_cols = NULL,
  ...
)
```

Arguments

data	Data frame
id_col	Primary key column
threshold	Numeric. Missing rate threshold (default 0.05 = 5%)
exclude_cols	Character vector of columns to exclude from check
...	Reserved

Value

A check_result object

check_missing_ids *Check for missing primary IDs*

Description

A.03: Records with NA or empty ID field.

Usage

```
check_missing_ids(data, id_col, ...)
```

Arguments

data	Data frame
id_col	Primary key column
...	Reserved

Value

A check_result object

check_name_format *Check name format validity*

Description

O.04: Flags names that are NA/empty, only digits, known test values, or single characters. Useful for catching enumerator shortcuts and data entry mistakes in respondent name fields.

Usage

```
check_name_format(data, id_col, name_col, ...)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`name_col`	Character. Name of the name column to check
`...`	Reserved for future use

Value

A check_result object

check_negative_values *Check for negative values*

Description

D.09: Flag negative values in columns that should be non-negative (e.g., counts, areas, quantities).

Usage

```
check_negative_values(data, id_col, num_col, ...)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`num_col`	Character. Name of the numeric column to check
`...`	Reserved for future use

Value

A check_result object

check_other_specify *Check "other (specify)" write-in responses*

Description

O.01: Extracts all unique "other (specify)" write-in responses for review. Flags records that have non-empty other_col values so they can be recoded or validated during data cleaning.

Usage

```
check_other_specify(data, id_col, other_col, ...)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`other_col`	Character. Name of the "other specify" text column
`...`	Reserved for future use

Value

A check_result object

check_outliers_iqr *Check outliers using IQR method*

Description

D.01: Flag values below Q1 - multiplier*IQR or above Q3 + multiplier*IQR.

Usage

```
check_outliers_iqr(data, id_col, num_col, multiplier = 1.5, ...)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`num_col`	Character. Name of the numeric column to check
`multiplier`	Numeric. IQR multiplier for fence calculation (default 1.5)
`...`	Reserved for future use

Value

A check_result object

`check_outliers_mad` *Check outliers using Modified Z-score (MAD method)*

Description

D.03: Flag values with absolute modified Z-score exceeding a threshold. Modified Z-score: M_i = 0.6745 * (x_i - median) / MAD.

Usage

```
check_outliers_mad(data, id_col, num_col, threshold = 3, ...)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`num_col`	Character. Name of the numeric column to check
`threshold`	Numeric. Modified Z-score threshold for flagging (default 3)
`...`	Reserved for future use

Value

A check_result object

`check_outliers_zscore` *Check outliers using robust Z-score*

Description

D.02: Flag values with absolute Z-score exceeding a threshold. Uses robust statistics: median and MAD instead of mean and SD.

Usage

```
check_outliers_zscore(data, id_col, num_col, threshold = 3, ...)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`num_col`	Character. Name of the numeric column to check
`threshold`	Numeric. Z-score threshold for flagging (default 3)
`...`	Reserved for future use

Value

A check_result object

check_phone_format *Check phone number format*

Description

O.05: Validates phone numbers by extracting digits and checking length bounds. Optionally validates country code prefix.

Usage

```
check_phone_format(
  data,
  id_col,
  phone_col,
  country_code = NULL,
  min_digits = 8,
  max_digits = 15,
  ...
)
```

Arguments

data	Data frame of survey submissions
id_col	Character. Name of the primary key column
phone_col	Character. Name of the phone number column
country_code	Character or NULL. Expected country code prefix (e.g., "+226"). If provided, numbers not starting with this prefix (after stripping non-digits except leading +) are flagged.
min_digits	Integer. Minimum number of digits (default 8)
max_digits	Integer. Maximum number of digits (default 15)
...	Reserved for future use

Value

A check_result object

check_photo_attachment *Check photo attachment completeness*

Description

N.06: Flag records where the photo column is missing, empty, or contains a placeholder value such as "none".

Usage

```
check_photo_attachment(data, id_col, photo_col, ...)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`photo_col`	Character. Name of the photo attachment column
`...`	Reserved for future use

Value

A check_result object

`check_respondent_eligibility`
Check respondent eligibility

Description

A.12: Flag records where the respondent does not meet eligibility criteria based on age and/or gender.

Usage

```
check_respondent_eligibility(
  data,
  id_col,
  age_col = NULL,
  min_age = 18,
  gender_col = NULL,
  eligible_gender = NULL,
  ...
)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`age_col`	Character or NULL. Name of the respondent age column (default NULL)
`min_age`	Numeric. Minimum eligible age (default 18)
`gender_col`	Character or NULL. Name of the gender column (default NULL)
`eligible_gender`	Character vector or NULL. Acceptable gender values (default NULL)
`...`	Reserved for future use

Value

A check_result object

check_response_entropy *Check response entropy (low diversity detection)*

Description

M.06: For each survey, compute normalized Shannon entropy across check_cols. Low entropy indicates suspiciously low response diversity, which may indicate fabricated data.

Usage

```
check_response_entropy(data, id_col, check_cols, min_entropy_ratio = 0.3, ...)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`check_cols`	Character vector. Columns to compute entropy over
`min_entropy_ratio`	Numeric. Minimum normalized entropy threshold; surveys below this are flagged (default 0.3)
`...`	Reserved for future use

Details

H = -sum(p * log2(p)) normalized by log2(n_categories).

Value

A check_result object

check_roster_completeness *Check roster completeness*

Description

C.07: Flag records where roster member count does not match the reported household size.

Usage

```
check_roster_completeness(data, id_col, hh_size_col, member_count_col, ...)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`hh_size_col`	Character. Name of the reported household size column
`member_count_col`	Character. Name of the actual roster member count column
`...`	Reserved for future use

Value

A check_result object

`check_skip_pattern` *Check skip pattern consistency*

Description

C.03: Verify that conditional questions follow skip logic.

Usage

```
check_skip_pattern(
  data,
  id_col,
  parent_col,
  child_col,
  parent_value,
  expect_child_na = TRUE,
  ...
)
```

Arguments

`data`	Data frame
`id_col`	Primary key column
`parent_col`	Parent (trigger) column name
`child_col`	Child (conditional) column name
`parent_value`	Value of parent that triggers the skip
`expect_child_na`	Logical. If TRUE, child should be NA when parent == parent_value
`...`	Reserved

Value

A check_result object

`check_speed_violations` *Check speed violations based on total survey duration*

Description

N.03: Flag surveys where total duration is below the expected minimum, computed as `total_questions * min_seconds_per_question`. This check works on the main survey data (one row per survey), not text audit data.

Usage

```
check_speed_violations(
  data,
  id_col,
  total_questions,
  total_duration_col,
  min_seconds_per_question = 5,
  ...
)
```

Arguments

`data`	Data frame of survey submissions (one row per survey)
`id_col`	Character. Name of the primary key column
`total_questions`	Integer. Total number of questions in the survey
`total_duration_col`	Character. Name of the total duration column (seconds)
`min_seconds_per_question`	Numeric. Minimum expected seconds per question (default 5)
`...`	Reserved for future use

Value

A check_result object

`check_straightlining` *Check for straightlining (identical response patterns)*

Description

B.09: For each survey, check what proportion of responses across check_cols have the same value. Flag surveys where the proportion of identical responses exceeds max_identical_rate.

Usage

```
check_straightlining(data, id_col, check_cols, max_identical_rate = 0.8, ...)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`check_cols`	Character vector. Columns to inspect for straightlining
`max_identical_rate`	Numeric. Maximum proportion of identical responses before flagging (default 0.8)
`...`	Reserved for future use

Value

A check_result object

`check_survey_duration` *Check survey duration*

Description

F.01: Flag surveys shorter than min_duration or longer than max_duration.

Usage

```
check_survey_duration(
  data,
  id_col,
  duration_col,
  min_duration = 10,
  max_duration = 120,
  ...
)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`duration_col`	Character. Name of the column containing duration in minutes
`min_duration`	Numeric. Minimum acceptable duration in minutes (default 10)
`max_duration`	Numeric. Maximum acceptable duration in minutes (default 120)
`...`	Reserved for future use

Value

A check_result object

`check_survey_end_before_start` *Check survey end time before start time*

Description

F.04: Flag surveys where the end timestamp is before the start timestamp.

Usage

```
check_survey_end_before_start(data, id_col, start_col, end_col, ...)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`start_col`	Character. Name of the survey start time column
`end_col`	Character. Name of the survey end time column
`...`	Reserved for future use

Value

A check_result object

`check_survey_tracking` *Check survey tracking against targets per stratum*

Description

A.10: Flag strata where actual survey count is below the target.

Usage

```
check_survey_tracking(data, id_col, strata_col, target_per_stratum, ...)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`strata_col`	Character. Name of the stratification column
`target_per_stratum`	Named numeric vector (stratum -> target) or a data frame with columns matching strata_col and "target"
`...`	Reserved for future use

Value

A check_result object

---

check_text_audit_duration — *Check text audit question-level duration*

Description

N.01: Flag question-survey combinations where the time spent on a question is less than `min_seconds`. Text audit data is expected in long format: one row per question per survey.

Usage

```
check_text_audit_duration(
  data,
  id_col,
  question_col,
  duration_col,
  min_seconds = 3,
  ...
)
```

Arguments

`data`	Data frame of text audit entries (long format)
`id_col`	Character. Name of the survey ID column
`question_col`	Character. Name of the question identifier column
`duration_col`	Character. Name of the duration column (in seconds)
`min_seconds`	Numeric. Minimum acceptable seconds per question (default 3)
`...`	Reserved for future use

Value

A check_result object

---

check_text_audit_sequence — *Check text audit question sequence for backward jumps*

Description

N.02: Detect question revisiting by checking if the question order has backward jumps within each survey. A backward jump means the order value decreases, indicating the enumerator revisited an earlier question.

Usage

```
check_text_audit_sequence(data, id_col, question_col, order_col, ...)
```

Arguments

`data`	Data frame of text audit entries (long format)
`id_col`	Character. Name of the survey ID column
`question_col`	Character. Name of the question identifier column
`order_col`	Character. Name of the order/sequence column (numeric)
`...`	Reserved for future use

Value

A check_result object

`check_text_duplicates` *Check for duplicated text responses (copy-paste detection)*

Description

O.03: Flags identical text responses across different surveys. Only considers text values with at least min_length characters to avoid false positives on short common answers.

Usage

```
check_text_duplicates(data, id_col, text_col, min_length = 10, ...)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`text_col`	Character. Name of the text column to check
`min_length`	Integer. Minimum text length to consider (default 10)
`...`	Reserved for future use

Value

A check_result object

check_text_length — *Check text response length*

Description

O.02: Flags text responses that are too short or too long. Responses shorter than min_length or longer than max_length characters are flagged. NA and empty strings are excluded from flagging.

Usage

```
check_text_length(
  data,
  id_col,
  text_col,
  min_length = 3,
  max_length = 500,
  ...
)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`text_col`	Character. Name of the text column to check
`min_length`	Integer. Minimum acceptable length (default 3)
`max_length`	Integer. Maximum acceptable length (default 500)
`...`	Reserved for future use

Value

A check_result object

check_variance_ratio — *Check variance ratio by enumerator*

Description

M.08: For each numeric variable, compute the ratio of between-enumerator variance to within-enumerator variance. A high ratio suggests enumerator identity explains too much of the variability, possibly indicating fabrication or systematic bias.

Usage

```
check_variance_ratio(data, id_col, enum_col, num_cols, max_ratio = 2, ...)
```

Arguments

`data`	Data frame of survey submissions
`id_col`	Character. Name of the primary key column
`enum_col`	Character. Name of the enumerator ID column
`num_cols`	Character vector. Numeric columns to test
`max_ratio`	Numeric. Maximum acceptable between/within variance ratio (default 2)
`...`	Reserved for future use

Value

A check_result object

`export_to_csv` *Export check results to CSV files*

Description

Writes summary and flagged records as separate CSV files.

Usage

```
export_to_csv(report, output_dir, prefix = "hfc_report", ...)
```

Arguments

`report`	A list of check_result objects
`output_dir`	Character. Directory for output files
`prefix`	Character. File name prefix (default "hfc_report")
`...`	Reserved for future use

Value

Invisible list of file paths created

`export_to_excel` *Export check results to an Excel workbook*

Description

Creates a multi-sheet Excel file with check results, flagged records, and summary statistics. Requires the openxlsx2 package.

Usage

```
export_to_excel(report, data, id_col, path, ...)
```

Arguments

`report`	A list of check_result objects (e.g., from running checks)
`data`	Original data frame (for extracting flagged rows)
`id_col`	Character. ID column name
`path`	Character. Output file path (.xlsx)
`...`	Reserved for future use

Value

Invisible path to the created file

`is_check_result` *Test if an object is a check_result*

Description

Test if an object is a check_result

Usage

```
is_check_result(x)
```

Arguments

`x`	Object to test

Value

Logical

`new_check_result` *Create a standardized check result*

Description

Every check function in afcestDataCheck returns this structure.

Usage

```
new_check_result(
  check_name,
  check_category,
  n_flagged = 0L,
  n_total = 0L,
  flagged_ids = character(),
  flag_reason = character(),
  severity = c("error", "warning", "info"),
  summary_stat = list()
)
```

Arguments

check_name	Character. Unique check identifier (e.g., "A01_duplicate_id")
check_category	Character. Category (e.g., "identification", "enumerator")
n_flagged	Integer. Number of flagged records
n_total	Integer. Total records examined
flagged_ids	Character vector. IDs of flagged records
flag_reason	Character vector. Reason per flagged ID
severity	Character. One of "error", "warning", "info"
summary_stat	List. Check-specific summary statistics

Value

A list of class "check_result"

---

normalize_survey_data *Normalize survey data to standard column names*

Description

Detects or accepts a survey platform and renames platform-specific columns to a standard set: .id, .enum, .date, .start, .end, .duration, .latitude, .longitude. Computes .duration from .start and .end if both exist.

Usage

```
normalize_survey_data(
  data,
  platform = c("auto", "kobo", "surveycto", "odk", "survey_solutions", "cspro"),
  id_col = NULL,
  enum_col = NULL,
  date_col = NULL,
  start_col = NULL,
  end_col = NULL,
  gps_col = NULL,
  ...
)
```

Arguments

data	Data frame of raw survey submissions
platform	Character. One of "auto", "kobo", "surveycto", "odk", "survey_solutions", "cspro". Default "auto" detects from column names.
id_col	Character or NULL. Explicit column name for primary ID
enum_col	Character or NULL. Explicit column name for enumerator
date_col	Character or NULL. Explicit column name for submission date
start_col	Character or NULL. Explicit column name for start time
end_col	Character or NULL. Explicit column name for end time
gps_col	Character or NULL. Explicit column name for GPS data
...	Reserved for future use

Value

A data frame with standard columns prepended, class "normalized_survey"

---

parse_gps_string — *Parse GPS string into structured components*

Description

Parses the "lat lon alt accuracy" space-separated format commonly used by KoBo Toolbox and ODK into a tibble with named columns.

Usage

```
parse_gps_string(gps_string)
```

Arguments

`gps_string`	Character vector. GPS strings in "lat lon alt accuracy" format

Value

A tibble with columns: latitude, longitude, altitude, accuracy

---

print.adc_report — *Print an adc_report summary*

Description

Print an adc_report summary

Usage

```
## S3 method for class 'adc_report'
print(x, ...)
```

Arguments

`x`	An adc_report object
`...`	Ignored

print.check_result *Print a check result*

Description

Print a check result

Usage

```
## S3 method for class 'check_result'
print(x, ...)
```

Arguments

x	A check_result object
...	Ignored

read_config *Read and validate a project configuration file*

Description

Read and validate a project configuration file

Usage

```
read_config(path)
```

Arguments

path	Path to YAML config file

Value

A validated config list of class "adc_config"

`recode_other_specify` *Recode "other specify" responses*

Description

Q.03: Apply a recode map to free-text "other (specify)" responses, mapping raw text values to standardized categories.

Usage

```
recode_other_specify(
  data,
  id_col,
  other_col,
  recode_map,
  target_col = NULL,
  ...
)
```

Arguments

`data`	Data frame containing the other-specify column
`id_col`	Character. Primary key column name (used for traceability)
`other_col`	Character. Name of the other-specify column to recode
`recode_map`	Named character vector. Names are the original text values (matched case-insensitively after trimming whitespace), values are the new categories. Example: `c("maiz" = "maize", "riz paddy" = "rice")`
`target_col`	Character or NULL. If provided, write recoded values to this column (creating it if necessary). If NULL, modify other_col in place.
`...`	Reserved for future use

Details

This is a data modification function, NOT a check function. It does not return a check_result object.

Value

The modified data frame

`run_all_checks` *Run all enabled checks from a configuration*

Description

This is the main entry point for the package. It reads a config, normalizes the data, and runs all enabled checks.

Usage

```
run_all_checks(
  data,
  config,
  sampling_frame = NULL,
  backcheck_data = NULL,
  verbose = TRUE,
  ...
)
```

Arguments

data	Data frame of survey submissions
config	Either a path to a YAML config file or an adc_config object
sampling_frame	Optional character vector of valid IDs
backcheck_data	Optional data frame for back-check comparisons
verbose	Logical. Show progress messages (default TRUE)
...	Reserved

Value

A list of class "adc_report" containing all check results

---

summary.adc_report *Summarize an adc_report*

Description

Returns the summary tibble produced by `bind_check_results()`.

Usage

```
## S3 method for class 'adc_report'
summary(object, ...)
```

Arguments

object	An adc_report object
...	Ignored

Value

A tibble summarizing all checks